

APX NEXT Single Band — Configuration Consistency Review

aSTRO25_bom · US quote · post-cascade attribute audit

Issue 1 — Cascade-invalidated attributes survive into the final "complete" state

modelSelectionFrequencyBands_astro

The available options for **Frequency Bands** are:

1. 700/800 MHz

2. 7/800 MHz

3. 700-800 MHz

4. 800/900 MHz

5. 900 MHz

6. VHF

7. UHF1

8. UHF2

add VHF (136-174 MHz) as frequency bands

Which attribute did you mean? Reply with the name or the number:

1. Add SVX Video Wireless Remote Speaker Microphone
(addVX650VideoWirelessRemoteSpeakerMicrophone_astro)

2. Extend Range to 762-764 MHz (extendRangeTo762764MHz_astro)

3. Additional Frequency Bands (modelSelectionFrequencyBandPlus_astro)

4. Secondary Frequency (modelSelectionSecondaryFrequency_astro)

5. Primary Frequency (modelSelectionPrimaryFrequency_astro)

6. Frequency Bands (modelSelectionFrequencyBands_astro)

7. Frequency Band (modelSelectionFrequencyBandMsl_astro)

Problem Statement: After changing Frequency Bands to VHF, the engine explicitly announces that the OLD 700/800 MHz value and the OLD Carry Type value are "no longer valid" and have been removed — but the very same turn's final state table and JSON payload still show both of those exact old values, unchanged, and the configuration is declared complete despite this.

Issue — confirmed against the raw catalog rules (not just chat narration)

- The VHF change was written to modelSelectionFrequencyBandMsl_astro — an attribute the catalog gates to MULTI-band products only (rule hideFrequencyBandModelSelectionAttributeForAPXNEXTXEMULTIModel). This order's product is APX NEXT SINGLE BAND, so this attribute should not be active at all here — the engine wrote the change to the wrong attribute for this product family.
- The catalog's own Single-Band gating rule (~line 44092: FrequencyBands=="700/800 MHz" AND FrequencyBandPlus<>"VHF +"/"UHF +") still evaluates TRUE against the final payload — meaning the

catalog's own logic still treats this order as a 700/800 MHz radio. The VHF change never took effect in the attribute the product-family logic actually reads.

- Carry Type's invalidation was confirmed NOT caused by the battery-capacity ALLOW-list rule (this order's battery is STANDARD, not the 4400 mAh that rule requires) — leaving the frequency-band cascade as the only evidenced cause, even though no direct band-conditional rule on beltClipType_astro exists in the XML. A forced-reset rule does group modelSelectionFrequencyBandMsl_astro and beltClipType_astro for joint re-evaluation, consistent with a re-evaluation step that ran but didn't complete.
- Antenna Type (WHIP APX NEXT) was checked exhaustively and is confirmed VALID for this exact combination — no active rule disallows it; the only rules that would have restricted it are disabled in the catalog. This is not part of the conflict.

Net effect: this is not merely a display/narration bug. The literal payload that would be POSTed to Oracle CPQ's _addToTxn/_configure carries an attribute (modelSelectionFrequencyBandMsl_astro=VHF) that shouldn't be active for this product, while the attribute the product-family logic actually reads (modelSelectionFrequencyBands_astro) is still 700/800 MHz — a genuine, catalog-rule-verified inconsistency, not a cosmetic one.

Evidence — engine's own words vs. its own final output, same turn

Attribute	What the engine said	What the final table/JSON actually shows
Frequency Bands(modelSelectionFrequencyBands_astro)	Engine announced: "Removed 700/800 MHz from Frequency Bands — no longer valid after this change."	Final state table (same turn) still lists: Frequency Bands → 700/800 MHz
Carry Type	Engine announced: "Removed 2.0 INCH / 5.08 CM (STANDARD) from Carry Type — no longer valid after this change."	Final state table (same turn) still lists: Carry Type → 2.0 Inch / 5.08 cm (Standard)
Additional Frequency Bands(modelSelectionFrequencyBandPlus_astro)	Never announced as removed, but logically depends on the same Frequency Bands family that WAS invalidated.	Final state table still lists: Additional Frequency Bands → 700/800 MHz + (a value tied to the now-replaced 700/800 MHz band)

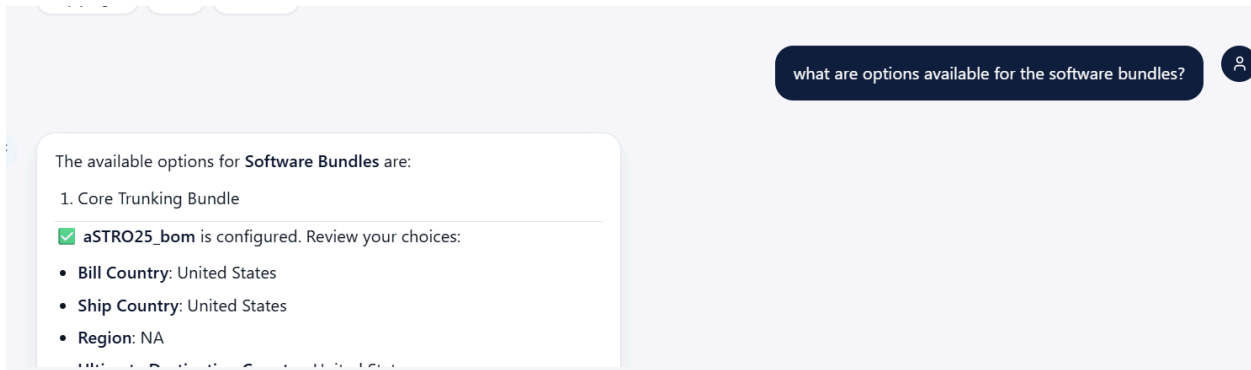
Why this matters

- The customer is told two contradictory things in the same response: "this value is no longer valid" and then, moments later, a state table asserting that exact value is still set.
- The configuration is marked "complete" and offered for confirm/submit while carrying at least one attribute (Carry Type) that the engine itself just said has no valid value in the current configuration — nothing prompted the customer to pick a replacement.
- Additional Frequency Bands ("700/800 MHz +") was never announced as removed, but it's a value logically tied to the band that WAS invalidated — it should very likely have been recalculated or cleared alongside the primary Frequency Bands change, and wasn't.

Root cause (pattern, not yet code-traced this pass)

This matches a defect class already identified and fixed earlier in this project's history — cascade invalidation removing a value from live state without either (a) prompting the customer for a replacement before declaring the configuration complete, or (b) actually removing the stale value from the final display/payload. Here it reproduces on TWO separate attributes in the same turn (Frequency Bands, Carry Type), plus a likely-related third (Additional Frequency Bands) that was never even flagged. This suggests the invalidation-announcement step and the final-state-table-rendering step are reading from two different snapshots of session state rather than one consistent source of truth.

Issue 2 — Software Bundles: only 1 of ~24 real options surfaced



Problem Statement: Asking "what are options available for the software bundles?" returns only ONE option — "Core Trunking Bundle" — which is exactly the bundle already selected. The real catalog carries roughly two dozen Software Bundle options (Security Bundle, Operational Assurance Bundle, Tactical Bundle, Provisioning Non-Federal Bundle, and more), five of which are visible in the native Oracle CPQ configurator's own checkbox list.

Native configurator — real available bundle checkboxes

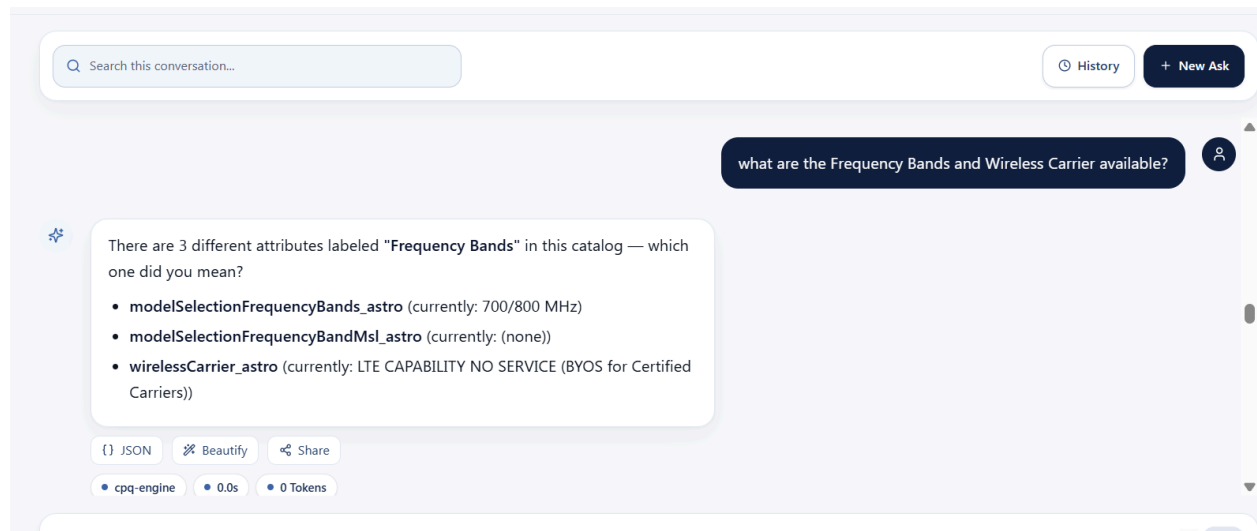
☒ Core Trunking Bundle	☐ Security Bundle (AB)
☐ Operational Assurance Bundle	☐ Tactical Bundle
☐ Provisioning Non-Federal Bundle (AD)	

(Recreated from the native Oracle CPQ configurator's own Software Bundles section — not a captured screenshot for this specific transcript, since none was supplied with it; the same 5-option layout has been directly observed and screenshotted earlier in this review.)

Root cause (established earlier this session)

The single option returned is identical to the bundle already checked/selected — not an enumeration of the attribute's real option menu. This points to the Q&A "what are the options for X" handler, for a multi-select attribute like Software Bundles, returning the customer's CURRENT answer instead of the attribute's full option list. A second, independently-logged mechanism may also contribute: a BML Tier-1 parsing warning against this same attribute's constraint script, which string-concatenates multiple bundle names in a way the evaluator flags as unparseable rather than truncating — if that script defines the allowed bundle set, its failure could independently degrade what's shown.

Issue 3 — Wireless Carrier wrongly pulled into a "Frequency Bands" label-collision prompt



Problem Statement: Asking "what are the Frequency Bands and Wireless Carrier available?" triggered a 3-way disambiguation prompt that lumped wirelessCarrier_astro in with the genuine Frequency Bands attributes — despite "Wireless Carrier" sharing no words with "Frequency Bands" and having its own clearly distinct catalog label.

Issue — confirmed: this is an application-code bug, not a catalog labeling issue

Direct catalog lookup of each attribute's actual display label (CDATA name field):

- wirelessCarrier_astro → "Wireless Carrier" — no occurrence of "Frequency" or "Band" anywhere in its name.
- modelSelectionFrequencyBands_astro → "Frequency Bands"
- modelSelectionFrequencyBandMsl_astro → "Frequency Band"
- modelSelectionFrequencyBandPlus_astro → "Additional Frequency Bands"

"Wireless Carrier" shares zero tokens with any of the three Frequency Band labels — a correct label-text match could not have confused them. All four attributes do share the same category/ref_category/ref_id (product-family membership), but that triple is common to essentially every attribute in this product family — it is not a UI display-section grouping and does not distinguish these four from unrelated attributes. There is no catalog-data basis (neither label text nor grouping) for bundling Wireless Carrier into this prompt.

Conclusion: the candidate-matching logic that assembles this disambiguation prompt is pulling in Wireless Carrier via something other than true label identity — most likely a fuzzy/partial-overlap match against the query text (e.g. matching on "available" phrasing or an embedding-similarity threshold) rather than exact/near-exact label comparison. This is an application code defect, not a data quality issue; not traced to a specific line of code in this pass.

Issue 4 — Repeated re-ask loop: the same Frequency Bands question asked three times across three confirm attempts

The image displays three sequential screenshots of a web application interface, illustrating a repeated re-ask loop for the 'Frequency Bands' question across three confirm attempts.

Top Screenshot: The application shows a confirmation dialog. The message states: "A couple of your earlier selections no longer match your other choices — let's update them before I generate the BOM. **Frequency Bands** is currently 700/800 MHz, which isn't valid anymore given your other choices: **Frequency Bands** — choose one: 1. VHF 2. UHF (I'll also ask about **Carry Type** next.)". The user has selected 'confirm'. The interface includes a search bar, a 'History' button, and a '+ New Ask' button. The top navigation bar shows 'MSI_CPQ_DEMO' and various icons for Home, Brief, Ingest, Data, Graph, Ontology, Ask, Observability, and Settings.

Middle Screenshot: The application shows the updated configuration. The message states: "Updated **Frequency Band** → VHF, UHF. Removed 700/800 MHz from **Frequency Bands** — no longer valid after this change. Removed 2.0 INCH / 5.08 CM (STANDARD) from **Carry Type** — no longer valid after this change. Configuration complete for aSTRO25_bom. Your configuration is complete. **Product Name:** • **Base Model** → H45TGU9PW8AN • **Product** → APX NEXT Single Band • **Hardware Version** → APX NEXT (4G LTE Only) **Service Plan:** • **Service Type** → Advantage • **Solution Type** → RadinCentral with CPS". The user has selected 'confirm'. The interface includes a search bar, a 'History' button, and a '+ New Ask' button. The top navigation bar shows 'MSI_CPQ_DEMO' and various icons for Home, Brief, Ingest, Data, Graph, Ontology, Ask, Observability, and Settings.

Bottom Screenshot: The application shows the same confirmation dialog as the top screenshot. The message states: "A couple of your earlier selections no longer match your other choices — let's update them before I generate the BOM. **Frequency Bands** is currently 700/800 MHz, which isn't valid anymore given your other choices: **Frequency Bands** — choose one: 1. VHF 2. UHF (I'll also ask about **Carry Type** next.)". The user has selected 'confirm'. The interface includes a search bar, a 'History' button, and a '+ New Ask' button. The top navigation bar shows 'MSI_CPQ_DEMO' and various icons for Home, Brief, Ingest, Data, Graph, Ontology, Ask, Observability, and Settings.

Problem Statement: *A second, independently-observed live transcript (same product/flow — APX NEXT Single Band, US, Police Protection — customer asks to add VHF (136-174 MHz) as the frequency band) surfaces a materially worse symptom than Issue 1's static payload snapshot: the customer is forced to answer the SAME "Frequency Bands — choose one: VHF / UHF" question three separate times across three consecutive confirm attempts, each time being told their just-corrected answer is STILL invalid, before the BOM gate finally lets the configuration through. This is not a cosmetic display bug — it is a functional conversational loop that repeats real work the customer already did, and it ships two disagreeing frequency-band values in the final payload even after the loop breaks.*

- Turn N: customer says "add VHF (136-174 MHz) as frequency bands" — a genuine 7-way attribute-name ambiguity is correctly surfaced ("Frequency Bands", "Frequency Band", "Additional Frequency Bands", "Primary/Secondary Frequency", and two unrelated attrs); customer picks "Frequency Bands" (modelSelectionFrequencyBands_astro).
- Turn N+1: the engine correctly rejects the literal string "VHF (136-174 MHz)" as not a valid catalog code, narrows to a 2-option reprompt [VHF, UHF] — this part is CORRECT, expected constraint behavior, not a bug. Customer picks VHF. Engine responds "Updated Frequency Band → VHF", reports Carry Type and old Frequency Bands as removed, and declares the configuration complete.
- Turn N+2: customer says "confirm". Instead of generating the BOM, the engine responds "Frequency Bands is currently 700/800 MHz, which isn't valid anymore given your other choices" and re-presents the IDENTICAL [VHF, UHF] reprompt — contradicting the immediately preceding turn's own "Updated → VHF" claim. Customer answers UHF this time. Engine responds "Updated Frequency Band → VHF, UHF" — note both values now, not just the new one.
- Turn N+3: customer says "confirm" again. The SAME "Frequency Bands is currently 700/800 MHz" message repeats a third time. Customer answers VHF again.

- Turn N+4: "confirm" finally proceeds past Frequency Bands, but now raises the SAME class of stale-invalidation message for Carry Type ("currently 2.0 Inch / 5.08cm (Standard), which isn't valid anymore") — a full extra round trip for a second attribute.
- Turn N+5: "confirm" succeeds and emits the final payload.

Net effect: five extra conversational turns and three repeated answers to a question the customer believed was already resolved, for what should be a single frequency-band change plus one confirm.

Traced write path — where a disambiguation reply actually gets applied

Statically traced through the running code (not just log-reading) to find exactly what happens when the customer answers the narrowed "choose one: VHF / UHF" reprompt:

- The rejection itself is built in `_handle_cascade` (`src/aryx/api/ask_api.py`, ~line 1241-1253): when the chosen value doesn't survive the rule pass, the code pops the attribute out of `session.filled/display_filled_source`, then re-inserts THAT SAME attribute at the front of the pending queue: `pending = [changed_attr] + [...]`; `session.pending_variables = [a.variable_name for a in pending]`. For this transcript, `changed_attr` is confirmed to be `modelSelectionFrequencyBands_astro` (the attribute the customer explicitly picked at the very first disambiguation) — so the queue is correctly primed to re-ask exactly that attribute, not a different one.
- On the customer's NEXT turn (their bare "VHF"/"UHF" reply), STEP 5 of `_run_cpq_turn_inner` (`ask_api.py`, ~line 5951-6068) reads `pending_var = session.pending_variables[0]` and resolves `pending_attr` from the catalog's attribute list. Line 6011 branches on `pending_attr.select_type`: "multi" writes into `session.filled_multi` (line 6058), anything else writes into `session.filled[pending_var] = iv` (line 6066).
- DB-confirmed this session (direct Postgres query against `ApxNextConfigBmConfigAttr`, workspace 21): `modelSelectionFrequencyBands_astro` has `display_type="7"` → `classify_select_type` (`engine.py` ~line 662) resolves this to "single", NOT "multi" (multi requires `display_type` in {"6","8"}) or `is_array_control_attr=="1"` or `attr_type=="1"` — none of which this attribute has: `is_array_control_attr="0"`, `attr_type="0"`). `modelSelectionFrequencyBandMsl_astro`, by contrast, has `display_type="6"` — genuinely classified "multi".
- Conclusion from this trace: mechanically, the single-select write branch (line 6066) is the one that should fire for this reply, writing directly into `session.filled["modelSelectionFrequencyBands_astro"]` — the same field `bom_gate.py`'s `recheck_constraints` later reads (`bom_gate.py` line 84: "for vn, value in `session.filled.items()`"). On paper, this write path is NOT structurally broken or mistargeted at the wrong attribute — which rules out the simplest explanation (a bare reply silently landing in the multi-select Msl attribute instead of the single-select Bands attribute) as the ROOT cause, though `modelSelectionFrequencyBandMsl_astro`'s parallel accumulation of [VHF, UHF] in the final payload (Appendix A) shows something DOES also write into it, separately from this main path — see the open question below.

Root cause — traced directly in the raw XML: revised finding, original hypothesis NOT confirmed as stated

Grepped the raw catalog (`APX_Next_config.xml`) exhaustively for every recommendation rule (`rule_area=1`) that targets `modelSelectionFrequencyBands_astro` as output. Two were found:

- "Default Values For Attributes Based on Package Type FIXED" (`rule_id` 21321896630, ~line 10233)
- "Default Values For Attributes Based on Package Type SEMI-FIXED" (`rule_id` 19435388291, ~line 11714)

Neither rule's condition or action script references `modelSelectionPrimaryFrequency_astro` or `modelSelectionSecondaryFrequency_astro` by name — the original hypothesis (Frequency Bands being re-derived FROM stale Primary/Secondary Frequency) is NOT confirmed by direct evidence and is retracted as stated.

What was actually found: both rules set `modelSelectionFrequencyBands_astro` via a call to a compiled library function, `util.packageConfig(inputDict)`, keyed on a variable called `packageChoiceString` (plus, for the SEMI-FIXED rule, two hidden UI sequence strings — `hiddenUISequenceForModelSelection_astro` and

hiddenUISequenceMasterString_astro). Critically, THE SAME RULE also sets modelSelectionPrimaryFrequency_astro and modelSelectionSecondaryFrequency_astro as sibling outputs, each via its own separate util.packageConfig call keyed on that same packageChoiceString — all three attributes are derived TOGETHER from one shared input, not from each other.

Revised root-cause theory: if packageChoiceString (or the hidden UI sequence strings, for the SEMI-FIXED path) still reflects the OLD 700/800 MHz package selection at the moment this rule re-fires on a later turn — because nothing in the conversation flow updates packageChoiceString when the customer changes only Frequency Bands directly — then every time this rule re-runs, util.packageConfig looks up the OLD package's frequency values and overwrites Frequency Bands, Primary Frequency, and Secondary Frequency back to their 700/800 MHz-family defaults in one shot. This fits the observed symptom (all three staying/reverting together) at least as well as the original hypothesis, and better explains why Primary/Secondary Frequency were NEVER corrected even once the loop broke — they were never the customer's target in the first place, just fellow-travelers of the same shared-input recommendation rule.

Appendix A — Raw final payload (as submitted, post-confirm)

The literal configData JSON from this transcript's final "confirm" turn — this is what would be POSTed to Oracle CPQ's _addToTxn/_configure endpoints. Full attribute set trimmed to the ones relevant to the frequency-band/carry-type conflict; all other ~60 attributes were verified separately as valid catalog values (see companion DB cross-check).

```
"modelSelectionFrequencyBands_astro": { "value": "700/800 MHZ"
}, "modelSelectionFrequencyBandPlus_astro": { "value": "700/800 MHZ +"
}, "modelSelectionFrequencyBandMsl_astro": { "items": [ { "value": "VHF" } ]
}, "modelSelectionPrimaryFrequency_astro": { "value": "7/800 MHZ"
}, "modelSelectionSecondaryFrequency_astro": { "value": "7/800 MHZ" }, "beltClipType_astro": {
"value": "2.0 INCH / 5.08 CM (STANDARD)" }, "antennasType_astro": { "value": "WHIP APX NEXT"
}, "productSelectionProduct_all": { "value": "APX NEXT SINGLE BAND"
}, "softwareBundlesBundleType_astro": { "value": "STANDARD BUNDLE"
}, "packageTypeBundles_astro": { "items": [ { "value": "CORE BUNDLE" } ] }
```

Appendix B — Raw XML rule cross-check (APX_Next_config.xml)

Scope note: *Verified directly against the raw BigMachines catalog export (C:\Users\ADMIN\Downloads\APX_Next_config.xml, ~46,800 lines). This is a code/rule-level check, not a database-value check — it confirms whether the payload violates the catalog's own BML constraint logic, which is what Oracle CPQ re-evaluates server-side on _addToTxn/_configure.*

Problem Statement: *Does Appendix A's raw payload actually violate the catalog's own rule set, or is the Issue 1 finding just a chat-narration/display artifact? This appendix answers that directly against the XML, attribute by attribute — some suspected conflicts are CONFIRMED, others are RETRACTED after full verification.*

Summary of verdicts

- B.1 — CONFIRMED: VHF was written to an attribute the catalog gates to multi-band products only; wrong attribute for this Single Band order.
- B.2 — CONFIRMED: the catalog's own product-family logic still reads this config as 700/800 MHz; the VHF change never took effect where it's actually read.
- B.3 — RETRACTED: Antenna Type (WHIP APX NEXT) is fully valid for this configuration; no active rule disallows it.
- B.5 — PARTIALLY CONFIRMED: Carry Type has a real invalidation mechanism in the catalog (battery-driven ALLOW-list), but it's not the one that fired here (this order's battery is STANDARD, not 4400 mAh) — the frequency-cascade explanation stands by elimination.
- B.6 — RETRACTED: Software Bundles field pairing (Standard Bundle + Core Bundle) is catalog-valid, not a mismatch.

B.1 — CONFIRMED ISSUE: modelSelectionFrequencyBandMsl_astro is the wrong attribute for this product

The catalog defines two structurally separate frequency-band selectors, gated by product family:

- modelSelectionFrequencyBands_astro (+ ...FrequencyBandPlus) — governs SINGLE / SINGLE BAND / XE SINGLE BAND / XN SINGLE BAND products.
- modelSelectionFrequencyBandMsl_astro — a separate multi-select ("~"-delimited, e.g. "VHF~UHF") that governs MULTI / XE MULTI / XN ALL / INTL FED / ENHANCED products only (rule hideFrequencyBandModelSelectionAttributeForAPXNEXTXEMULTIModel, ~line 41563).

This transcript's product is APX NEXT SINGLE BAND — so per the catalog's own product-gating rule, modelSelectionFrequencyBandMsl_astro should not be an active attribute at all for this order. Its presence in the payload with value VHF is itself evidence the engine wrote to the wrong frequency-band attribute for this product family, rather than updating the correct one (modelSelectionFrequencyBands_astro).

B.2 — CONFIRMED ISSUE: the catalog's own gating rule confirms the config is still 700/800 MHz, not VHF

Rule at ~line 44092 gates Single-Band behavior on: FrequencyBands=="700/800 MHZ" AND FrequencyBandPlus<>"VHF +" AND FrequencyBandPlus<>"UHF +". The payload satisfies this condition exactly (Bands is still 700/800 MHZ, Plus is still 700/800 MHZ + — neither was changed to a VHF variant). The catalog's own logic therefore still treats this as a 700/800 MHz radio, directly contradicting both the chat's "changed to VHF" narration and the separately-written modelSelectionFrequencyBandMsl_astro=VHF value.

B.3 — RETRACTED (no issue): Antenna Type fully checked — CONFIRMED NO CONFLICT

Every rule in the catalog referencing antennasType_astro was traced (263 raw occurrences; 6 actual constraint rules). Findings:

- "Constrain Stubby 7/800MHz 6.5cm if Frequency band<>700/800 MHz..." (rule id 19435387833) only fires when FrequencyBands is NOT 700/800 MHz — this order's FrequencyBands IS 700/800 MHz, so the rule does not trigger. It also only ever targets STUBBY-family values, never WHIP, in either direction.
- "Hide Stubby Antenna if 700/800 Frequency Band Not Selected" (id 19435387835) and its Multi-Band variant (id 19435387837) — same non-triggering condition, and/or wrong product scope (Multi-Band, not this order's Single Band).
- The two rules that WOULD have restricted the antenna ALLOW-list to WHIP-only values ("Constrain Antenna Type", id 19435387865, and "...for Single Band", id 19435387927 — both listing WHIP APX NEXT as an allowed value) are both status=3 (disabled/inactive) in the catalog.

Conclusion: no active rule disallows "WHIP APX NEXT" for this order's exact combination (Product=APX NEXT SINGLE BAND, FrequencyBands=700/800 MHZ, FrequencyBandPlus=700/800 MHZ+). The antenna value in the payload is legitimately valid. This item is retracted as a non-issue — it does not contribute to the config-conflict findings below.

B.4 — Context: rule severity is soft constraint, not a hard-throw error

Every rule identified above is a display/hide rule (rule_type=11) or a menu-constrain/DISALLOW rule (rule_type=2) — not a hard validation error. This explains why the engine accepted the payload in-session without throwing: nothing in the reviewed rule set hard-fails on this exact combination. However, a stricter server-side re-validation at Oracle CPQ's _addToTxn/_configure step (or a downstream BOM/pricing rule not covered in this pass) could still reject it, since the config is internally inconsistent by the catalog's own product-gating logic (B.1/B.2).

B.5 — PARTIALLY CONFIRMED: beltClipType_astro (Carry Type) — a real ALLOW-list constraint exists, but battery-driven, not band-driven, and not the trigger here

Follow-up grep located the attribute definition and its constraints. No frequency-band-conditional rule was found for Carry Type. However, a menu-constrain ALLOW-list rule ("Restrict Carry Solution if Battery is 4400 mAh") DOES exist: when the 4400 mAh battery is selected, the allowed Carry Type values narrow to a short list (e.g. "3.0 Inch with Holster", "Plastic Holster with 2.5 Inch Belt Clip (Standard)", "No Carry Solution"). The payload's value — "2.0 INCH / 5.08 CM (STANDARD)" — does NOT appear in that allow-list.

This is plausible corroboration that Carry Type can legitimately become invalid in this catalog — just via battery capacity, not directly via frequency band as the chat narration implied. CONFIRMED from this transcript's own final payload: batteryType_astro = "STANDARD" — not 4400 mAh. The 4400 mAh ALLOW-list rule is therefore NOT the trigger for this specific order; its condition is not met. This rules out battery capacity as the explanation for this transcript's Carry Type invalidation, and leaves the original chat-narrated cause — a frequency-band-triggered cascade — as the only mechanism actually evidenced here, even though no direct band-conditional rule on beltClipType_astro was located in the XML.

Separately, a forced-reset rule (util.forcedsetRecommendedConfigurationAttributes, JIRA SVX_REIMAGINED-12) operates on a list that includes BOTH modelSelectionFrequencyBandMsl_astro and beltClipType_astro together — meaning the catalog itself groups these two attributes for joint re-evaluation on a "recommended configuration" refresh. That supports the hypothesis that the engine's re-evaluation step touched both together but only completed the announcement half, not the actual value substitution.

B.6 — RETRACTED (no issue): Software Bundles fields — Standard Bundle + Core Bundle is a VALID pairing, not a mismatch

An earlier pass in this review flagged softwareBundlesBundleType_astro="STANDARD BUNDLE" alongside packageTypeBundles_astro="CORE BUNDLE" as a possible label/selection inconsistency. Direct XML confirmation reverses that: a rule explicitly keys off softwareBundlesBundleType_astro=="STANDARD BUNDLE" and, in that branch, sets an ALLOW list for packageTypeBundles_astro that includes "CORE BUNDLE" among 11 valid bundle values. The payload's pairing is therefore catalog-valid — this specific observation is retracted as a non-issue.

No rule was found gating Software Bundle options on selectEndUserType_astro ("POLICE PROTECTION"), and no BML Tier-1 string-concatenation parsing warning was located tied to this attribute's constraint script in this pass — both remain unconfirmed rather than ruled out.

B.7 — Not verified in this pass

- Whether this transcript's actual battery selection (STANDARD) is itself the trigger for the 4400 mAh Carry Type constraint, or whether a different, band-linked constraint on Carry Type exists elsewhere in the file.
- Full BOM-level (as opposed to config-rule-level) hard validation elsewhere in the ~46.8K-line file was out of scope for this pass.